



PUCE
Pontificia Universidad
Católica del Ecuador

PUCE VIRTUAL

Facultad de Ingeniería
Ingeniería en Ciberseguridad

DESARROLLO DE PLATAFORMAS

T1S6 · Unidad 6

Informe técnico de SecureGuard
Seguridad, JWT, roles e integración
cliente-servidor

HTTPS · bcrypt · menor privilegio · MySQL

Autor

Alejandro David Carriel Zambrano

Proyecto SecureGuard · carpeta SEMANA 6

Quito — Ecuador · 2026

1. De qué va esta entrega

SecureGuard es el centro de incidentes que fui armando en la materia. Esta entrega no reemplaza la interfaz ni la base: protege la API que ya persistía en MySQL y conecta la pantalla a ese contrato. La pregunta de la unidad era si ocultar un botón basta. No basta. Un cliente puede armar el DELETE aunque el botón no exista, así que la regla quedó en el servidor.

2. Cómo corre la plataforma

Todo vive en la carpeta SEMANA 6. Docker mantiene el contenedor `mysql-incidentes` con la base `secureguard`. Prisma 7, con el adaptador MariaDB, lee `DATABASE_URL` desde `.env`. Express escucha por HTTPS en el puerto 3000 y también entrega el HTML, porque el interruptor HTTPS de Live Server impidió levantar la página. El certificado es local, generado con `selfsigned`. El navegador advierte y se continúa: en producción ese certificado lo pondría un proxy, no el código de la práctica. Lo dejo escrito porque la consigna pide HTTPS sin exigir una autoridad pública.

3. Capas

Las capas quedan separadas. `hash.js` genera y compara `bcrypt`. `token.js` firma un JWT de dos horas con `JWT_SECRET` y no mete la contraseña en el payload. `rol.js` exige el Bearer, vuelve a leer el usuario en MySQL y rechaza cuentas inactivas. El controlador valida título, descripción, severidad, tipo y fecha. El modelo solo escribe en Prisma. CORS, instalado como paquete, permite el origen `https://localhost:3000` y los de Live Server, con el encabezado `Authorization`. Un origen arbitrario no queda abierto con credenciales.

4. Roles

Los roles salen del principio de menor privilegio. El analista lee y crea. El supervisor lee y cambia estado. El administrador crea, edita y borra. Las tres cuentas se sembraron con `node src/seed.js`. `claves.txt` guarda las claves de la práctica; la tabla `Usuario` solo guarda el hash, que empieza por `$2`. Ocultar Eliminar en la vista del analista mejora la interfaz, pero el DELETE sigue respondiendo 403 si ese token se pega en Thunder Client.

5. Integración con el front

La integración usa un solo cliente `fetch`. `modelo.js` añade `Content-Type` y el Bearer guardado en `sessionStorage`. La vista pinta tarjetas y solo muestra Cambiar estado o Eliminar si el rol alcanza. El controlador anuncia carga, éxito y error en una región `aria-live`. Crear vuelve a pedir el listado, así que el código y la fecha los confirma el servidor. Si la sesión no existe, la página no inventa datos: pide ingresar.

6. Qué se probó

Registré ocho casos, más tres de la pantalla. Están en la carpeta `evidencias`, en la raíz del trabajo. **01-login-200** muestra el POST `/api/auth/login` del analista, estado 200, token y rol, sin devolver la clave. **02-sin-token-401** es el GET `/api/incidentes` sin `Authorization`. **03-analista-delete-403** es el DELETE de INC-002 con el analista. **04-titulo-corto-400** rechaza título de tres letras y descripción corta, con detalles por campo. **05-admin-crea-201** crea INC-005. **06-supervisor-patch-200** cambia ese caso a Contención en curso. **07-token-falso-401** rechaza un Bearer inventado. **08-despues-de-reiniciar** lista cuatro incidentes después de detener y volver a `npm start`: la fila no vivía en memoria. **09-login-pagina** es el formulario de acceso. **10-analista-sin-borrar** muestra el listado cargado desde MySQL sin botón Eliminar. **11-admin-con-botones** muestra Cambiar estado y Eliminar.

7. Limpieza, fallos y corrección

La limpieza acompaña a la validación. El servidor recorta correo y título, ignora un id enviado por el cliente y no copia un rol que venga en el body del incidente. La pantalla asigna el texto de las tarjetas sin interpretar HTML. Esas dos medidas cierran el caso en el que una descripción con etiquetas podría ejecutarse en el navegador o un campo extra podría cambiar permisos.

Live Server dejó de iniciar al activar HTTPS con cert y key vacíos; lo desactivé y el canal pasó a Express. El listado fallaba entre puertos hasta configurar cors. La migración init no tenía Usuario; hizo falta una segunda migración y `prisma generate`, porque el cliente viejo no conocía `prisma.usuario.hash.js` y `token.js` no los crea npm. `exigirToken` llegaba undefined y Express no arrancaba hasta exportarlo.

El fallo más útil fue el DELETE en 200 con el analista: el token se exigía, el rol no, y se borró INC-001. Añadí `exigirRol` en la ruta y el administrador recreó el caso. Ese error, la corrección y la captura del 403 son la evidencia de que la seguridad se probó, no solo se nombró.

Anexo. Evidencias

Ruta: SEMANA 6/evidencias. El informe no repite el token completo.

Archivo	Qué demuestra
01-login-200.png	Login 200, token y rol analista.
02-sin-token-401.png	GET sin Bearer.
03-analista-delete-403.png	Analista identificado, sin permiso de borrado.
04-titulo-corto-400.png	Validación de título y descripción.
05-admin-crea-201.png	Alta persistente INC-005.
06-supervisor-patch-200.png	Cambio de estado por supervisor.
07-token-falso-401.png	Firma inválida.
08-despues-de-reiniciar.png	Los cuatro casos siguen tras reiniciar Node.
09-login-pagina.png	Formulario de acceso en HTTPS.
10-analista-sin-borrar.png	Listado real, sin acción de admin.
11-admin-con-botones.png	Acciones visibles solo para el rol que el servidor permite.

Tabla 1. Capturas citadas en la carpeta evidencias.